南方科技大学
SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY

计算机组成原理
COMPUTER ORGANIZATION

本课件内容参考自David A. Patterson与John L. Hennessy的教材《Computer Organization and Design: The Hardware/Software Interface》官方配套课件。

zhangj4@sustech.edu.cn

Lecture 3

RISC-V Procedure

zhangj4@sustech.edu.cn

Recap

- Instruction set architecture
  - RISC vs. CISC
  - RISC-V/MIPS/ARM/x86
- Instructions
  - Arithmetic instruction: add, sub, …
  - Data transfer instruction: lw, sw, lh, sh, …
  - Logical instruction: and, or, …
  - Conditional branch: beq, bne, …
- Basic concepts
  - Operands: register vs. memory vs. immediate
  - Numeric representation: signed, unsigned, sign extension

Outline

- Procedure Call
- Non-Leaf Procedure

Procedure Call

Procedure Calling

- Six Steps of Calling a Function
  1. Put parameters in a place where the procedure can access them.
  2. Transfer control to the procedure.
  3. Acquire the storage resources needed for the procedure.
  4. Perform the desired task.
  5. Put the result value in a place where the calling program can access it.
  6. Return control to the point of origin, since a procedure can be called from several points in a program.
- We can think of a procedure like a spy
  - acquires resources → performs task → covers his tracks → returns back with desired result

Caller:

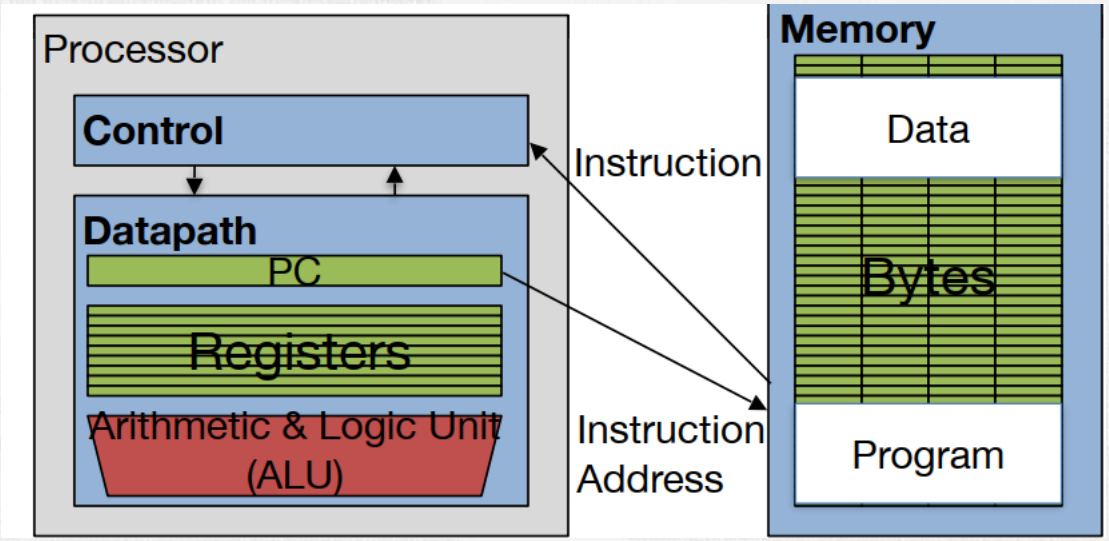
```
int x = 5;
int y = 3;
int z = sum (x , y);
x = x + 7;
…
```

callee:

```
int sum ( int a, int b)
{
    int c = a + b;
    return c;
}
```

Program Counter

- PC (program counter) is special internal register inside processor holding byte address of next instruction to be executed
- Instruction is fetched from memory, then control unit executes instruction using datapath and memory system, and updates program counter (default is add +4 bytes to PC, to move to next sequential instruction)



Procedure Calling

Caller:

```
int x = 5;
int y = 3;
int z = sum (x , y);
x = x + 7;
…
```

callee:

```
int sum ( int a, int b)
{
    int c = a + b;
    return c;
}
```

Data Memory

s0(x8)
s1(x9)
s2(x18)
a0(x10)
a1(x11)
a2(x12)
ra(x1)

if no enough registers

Register

	⋮
a0(x10)	~~5~~ 8
a1(x11)	3
	⋮
	⋮
	⋮

PC

Program Memory

x = 5;
y = 3;
int z = sum (x , y);
…
int sum (int a, int b)
{ int c = a + b;
return c; }

Where to Put Arguments and Return Values?

- Registers way faster than memory, so use them whenever possible
- Symbolic register names
  - a0-a7 (x10-x17) for function arguments, a0-a1 also for return values
  - ra (x1) for return address, used to save where a function is called from so we can get back
- If need extra space, use memory (the stack!)

RISC-V Registers and Convention

- ABI: Application Binary Interface, defines “Calling Convention” (will explain later)
- So going forward, no more x3, x6... type nomenclature

Register	ABI name	Description	Saved by Callee?
x0	zero	the constant value 0	N/A
x1	ra	Return address	No
x2	sp	Stack pointer	Yes
x3	gp	Global pointer	N/A
x4	tp	Thread pointer	N/A
x5 – 7	t0 – 2	Temporaries	No
x8	s0 / fp	Saved register/Frame pointer	Yes
x9	s1	Saved register	Yes
x10 – 17	a0 – 7	Function arguments/Return values	No
x18 – 27	s2 – 11	Saved registers	Yes
x28 – 31	t3 – 6	Temporaries	No

Transfer Control

- Procedure call: jump and link (the real order is link and jump)
  - jal ra, target #pseudo code: jal target
    - Address of following instruction (return address) put in ra
    - Jumps to target address
    - Used by Caller
- Procedure return: jump register
  - jalr zero, 0(ra) #pseudo code: jr ra
    - Similar to “Jump and Link” except in specification of target
    - Jump to ra and simultaneously saves the address of following instruction in zero register (value put into zero is meant to be throw away)
    - Used by Callee

Transfer Control

```
Caller:
        int x = 5;  // x in s0
        int y = 3;  // y in s1
        int z = sum (x , y);
        …
```

```
callee: // a in a0, b in a1, c in a0
        int sum ( int a, int b) {
            int c = a + b;
            return c;
        }
```

code address (in decimal) | Instructions

```
1000 addi a0,s0,0      # parse argument x → a
1004 addi a1,s1,0      # parse argument y → b
1008 jal sum           # ra=1012, goto sum
1012 …                 # next instruction
…
2000 sum: add a0,a0,a1
2004 jr   ra           # next instr at 1012…
```

Note: these are pseudo code

PC's decimal value: …→1000→1004→1008→2000→2004→1012→…

procedure call return

Transfer Control

```
Caller:
        int x = 5;  // x in s0
        int y = 3;  // y in s1
        int z = sum (x , y);
        …
```

```
callee: // a in a0, b in a1, c in a0
        int sum ( int a, int b) {
            int c = a + b;
            return c;
        }
```

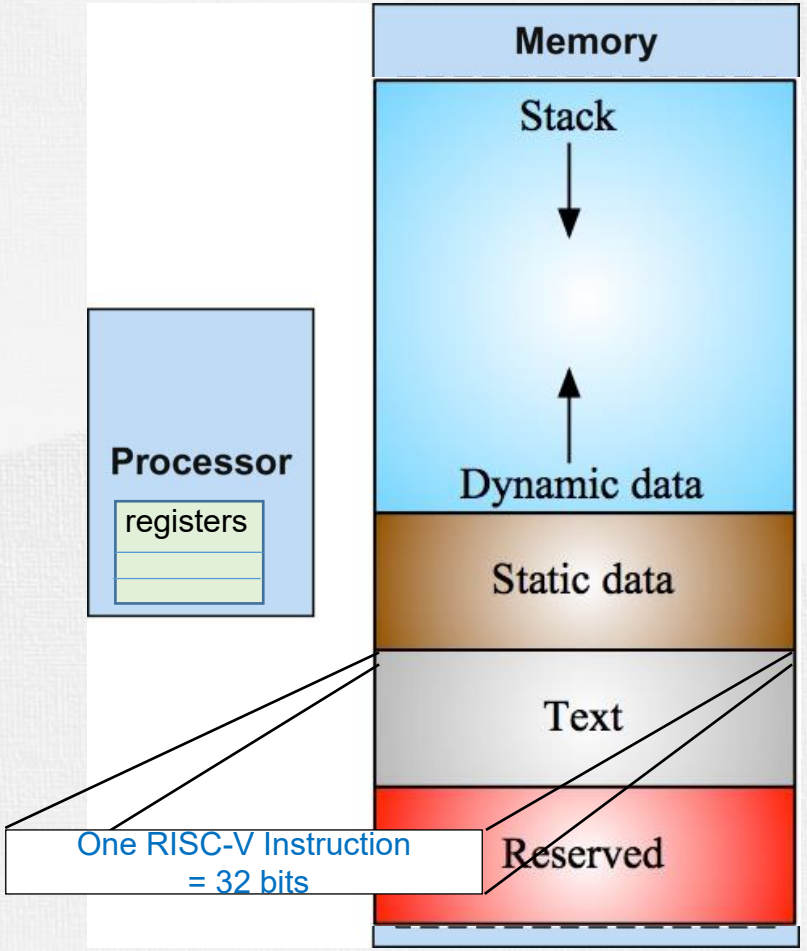
- Question: Why use jr to return? Why not use j ? such as:

```
1008 jal sum
1012 ret_point
…
2000 sum: add a0,a0,a1
2004 jr   ra              # jump to the return address
2004 j    ret_point       # working or not?
```

- Answer: sum might be called by many places, so we can't return to a fixed place. The calling proc to sum must be able to say "return here" somehow.

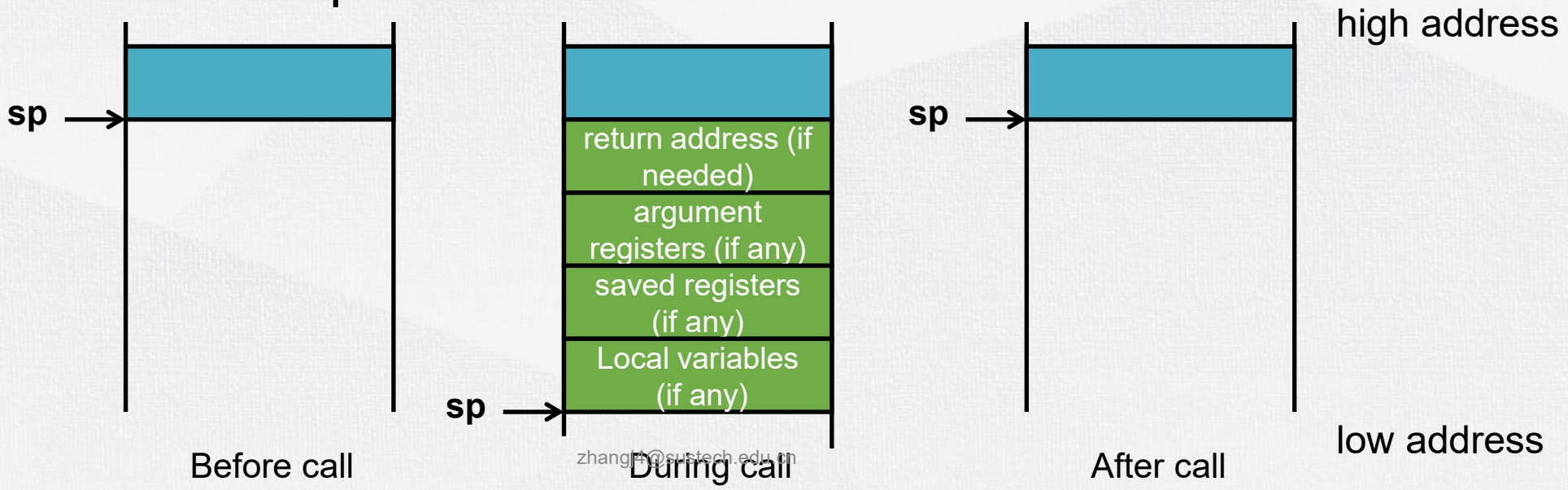
Recall the Memory Layout

- Text: Instructions (binary program code)
- Static data: global variables
  - e.g., static variables in C, constant arrays and strings
  - global pointer initialized to address allowing ±offsets into this segment
- Dynamic data: heap
  - E.g., malloc in C, new in Java
- Stack: automatic storage



Local storage for variables - Stack

- Use stack for automatic (local) variables that don't fit in registers
  - Push: placing data onto stack
  - Pop: removing data from stack
- sp (x2) (stack pointer) points to the last used place at the stack
  - Push decrements sp
  - Pop increments sp



zhangfh@sustech.edu.cn

Leaf Procedure Example

- Assume
  - the parameter variables g, h, i, and j correspond to the argument registers a0, a1, a2, and a3
  - f corresponds to s4
  - and use t0 and t1 for temporary value g+h and i+j

```
int  leaf_example (int g, int h, int i, int j)
{
    int f ;
    f = (g + h) – (i + j);
    return f;
}
```

```
leaf_example:
    addi    sp,  sp,  -12
    sw      t1, 8(sp)
    sw      t0, 4(sp)
    sw      s4, 0(sp)
    add     t0, a0, a1
    add     t1, a2, a3
    sub     s4, t0, t1
    add     a0, s4, zero
    lw      s4, 0(sp)
    lw      t1, 4(sp)
    lw      t0, 8(sp)
    addi    sp, sp, 12
    jr      ra
```

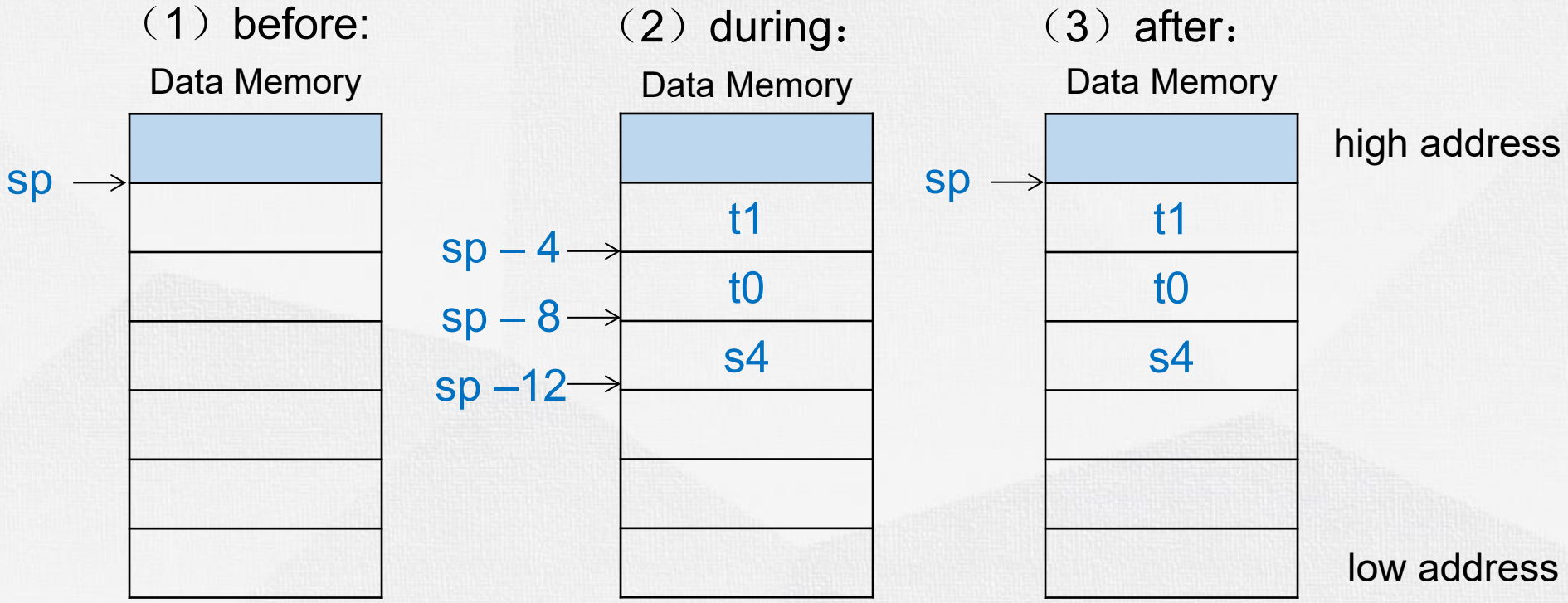
Save t0,t1,s0 (push)
Protect environment

Procedure body

Restore t0 t1 s0 (pop)

Return result

Stack Snapshots



To avoid too many memory operations:

t0 - t9: temporary registers are not preserved by the callee

s0 - s7: saved registers must be preserved by the callee if used

Calling Convention

- It is effectively a contract between functions
- caller-saved: Caller must protect the value if use
  - **a0~a7**: arguments and return values
  - **t0~t6**: Temporaries
  - **ra**: return address

- callee-saved registers: Callee must protect the values if use
  - **s0~s11**: Saved registers

Recall Leaf Procedure Example

- This is a “leaf procedure”
  - it calls no other function, so it’s a callee
  - no need to save a0~a4, t0 and t1 because they are caller saved
  - Instead of using s4, we can just use a0 for return value
- So the code can be optimized to:

```
leaf_example:
    add     t0, a0, a1
    add     t1, a2, a3
    sub     a0, t0, t1
    jr      ra
```

Caller saved:
- a0 ~ a7
- t0 ~ t6
- ra

Callee saved:
- s0 ~ s11

Non-Leaf Procedure

Non-Leaf Procedures

- Procedures that call other procedures (nested call)
  - Note: this could mean a function calling itself – recursion
- For nested call, caller needs to save on the stack:
  - Its return address (ra)
  - Any arguments and temporaries needed after the call
- Restore from the stack after the call

Non-Leaf Procedure Example

```
int   fact  (int n)
{
    if (n < 1)  return (1);
        else return (n * fact(n-1));
}
```

```
fact:
    addi    sp, sp, -8
    sw      ra, 4(sp)
    sw      a0, 0(sp)
    slti    t0, a0, 1
    beq     t0, zero, L1
    addi    a0, zero, 1
    addi    sp, sp, 8
    jr      ra
L1:
    addi    a0, a0, -1
    jal     fact
    addi    t1, a0, 0
    lw      a0, 0(sp)
    lw      ra, 4(sp)
    addi    sp, sp, 8
    mul     a0, a0, t1
    jr      ra
```

Compare n<1

Return 1

Run fact(n-1)

Return n*fact(n-1)

Notes:

The caller saves a0 and ra in its stack space.

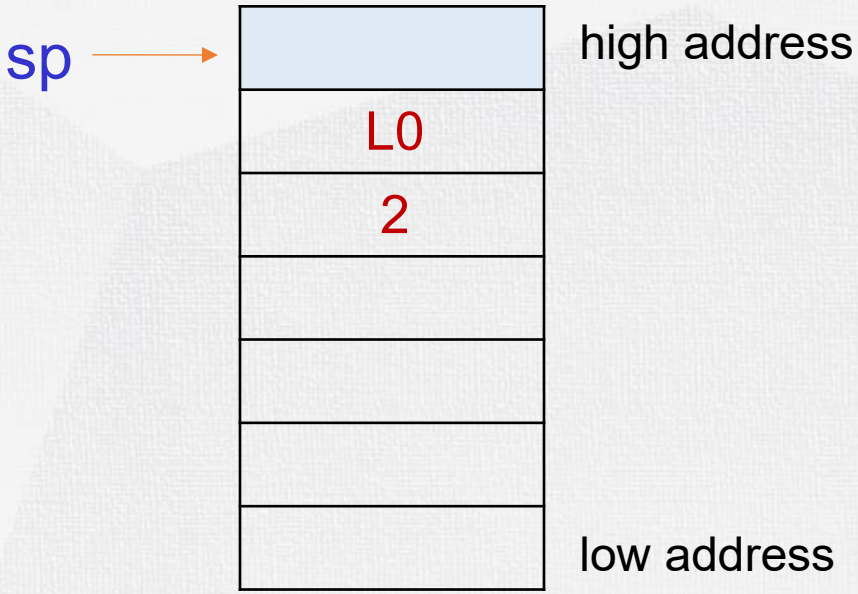
Temps are never saved.

Non-Leaf Procedure Example

```
fact:
    addi    sp, sp, -8
    sw      ra, 4(sp)
    sw      a0, 0(sp)
    slti    t0, a0, 1
    beq     t0, zero, L1
    addi    a0, zero, 1
    addi    sp, sp, 8
    jr      ra
L1:
    addi    a0, a0, -1
    jal     fact
    addi    t1, a0, 0
    lw      a0, 0(sp)
    lw      ra, 4(sp)
    addi    sp, sp, 8
    mul     a0, a0, t1
    jr      ra
```

a0=2

```
int   fact  (int n)
{
    if (n < 1)  return (1);
        else return (n * fact(n-1));
}
```

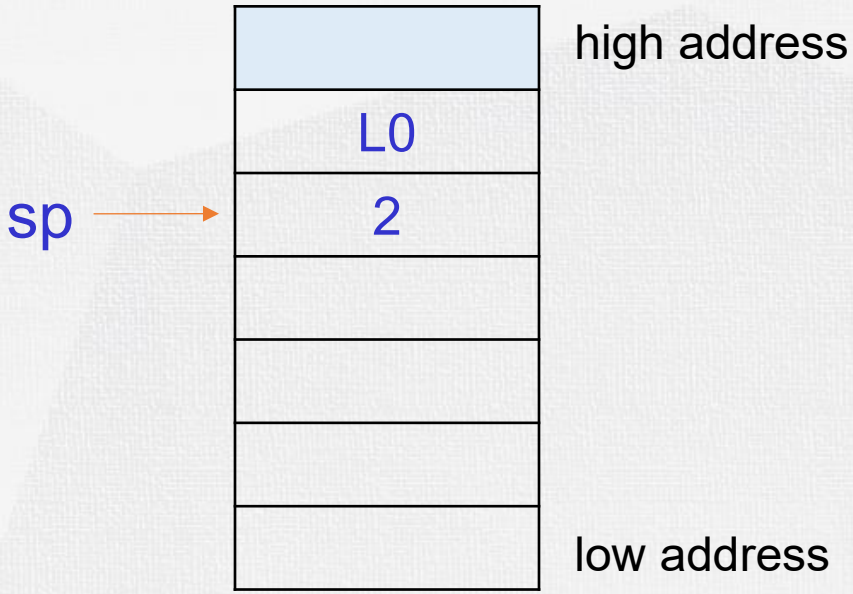


Non-Leaf Procedure Example

```
fact:
    addi    sp, sp, -8
    sw      ra, 4(sp)
    sw      a0, 0(sp)
    slti    t0, a0, 1
    beq     t0, zero, L1
    addi    a0, zero, 1
    addi    sp, sp, 8
    jr      ra
L1:
    addi    a0, a0, -1
    jal     fact
    addi    t1, a0, 0
    lw      a0, 0(sp)
    lw      ra, 4(sp)
    addi    sp, sp, 8
    mul     a0, a0, t1
    jr      ra
```

a0=2 t0=0

```
int   fact  (int n)
{
    if (n < 1)  return (1);
        else return (n * fact(n-1));
}
```

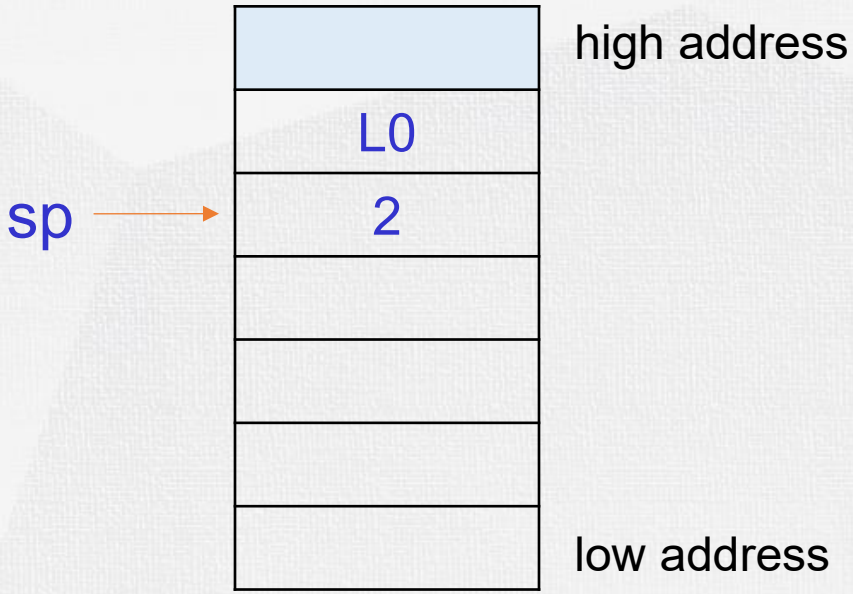


Non-Leaf Procedure Example

```
fact:
    addi    sp, sp, -8
    sw      ra, 4(sp)
    sw      a0, 0(sp)
    slti    t0, a0, 1
    beq     t0, zero, L1
    addi    a0, zero, 1
    addi    sp, sp, 8
    jr      ra
L1:
    addi    a0, a0, -1
    jal     fact
    addi    t1, a0, 0
    lw      a0, 0(sp)
    lw      ra, 4(sp)
    addi    sp, sp, 8
    mul     a0, a0, t1
    jr      ra
```

a0=1 t0=0

```
int   fact  (int n)
{
    if (n < 1)  return (1);
        else return (n * fact(n-1));
}
```

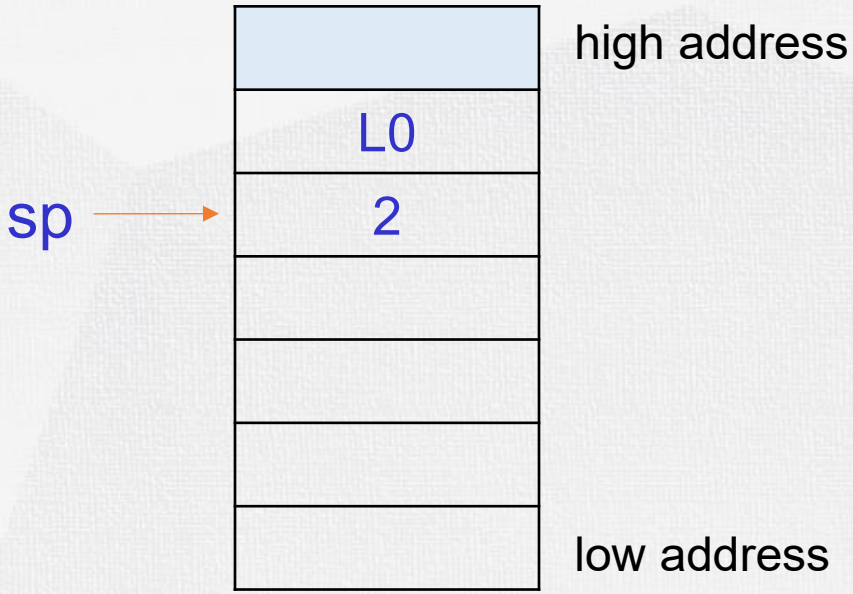


Non-Leaf Procedure Example

```
fact:
    addi    sp, sp, -8
    sw      ra, 4(sp)
    sw      a0, 0(sp)
    slti    t0, a0, 1
    beq     t0, zero, L1
    addi    a0, zero, 1
    addi    sp, sp, 8
    jr      ra
L1:
    addi    a0, a0, -1
    jal     fact
L2: addi    t1, a0, 0
    lw      a0, 0(sp)
    lw      ra, 4(sp)
    addi    sp, sp, 8
    mul     a0, a0, t1
    jr      ra
```

a0=1 t0=0 ra=L2

```
int  fact  (int n)
{
    if (n < 1)  return (1);
        else return (n * fact(n-1));
}
```

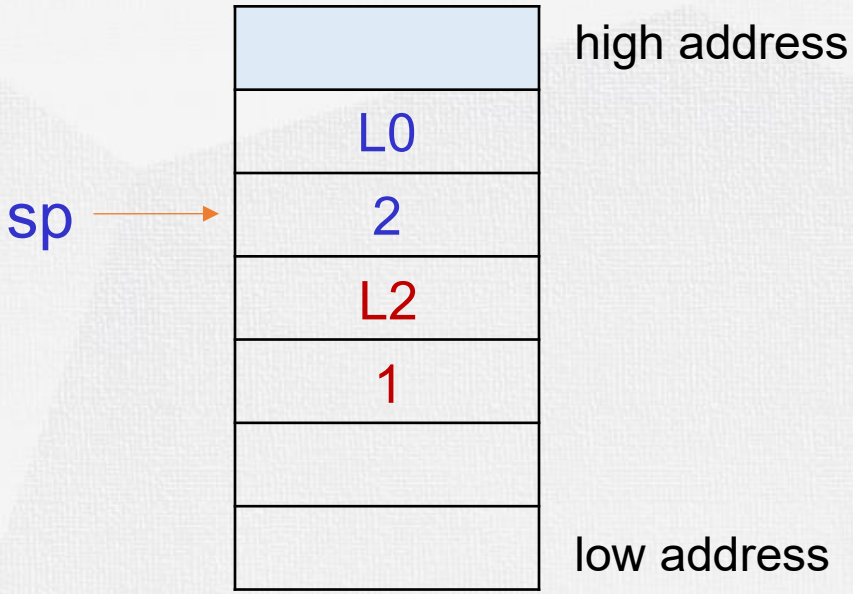


Non-Leaf Procedure Example

```
fact:
    addi    sp, sp, -8
    sw      ra, 4(sp)
    sw      a0, 0(sp)
    slti    t0, a0, 1
    beq     t0, zero, L1
    addi    a0, zero, 1
    addi    sp, sp, 8
    jr      ra
L1:
    addi    a0, a0, -1
    jal     fact
L2: addi    t1, a0, 0
    lw      a0, 0(sp)
    lw      ra, 4(sp)
    addi    sp, sp, 8
    mul     a0, a0, t1
    jr      ra
```

a0=1 t0=0 ra=L2

```
int  fact  (int n)
{
    if (n < 1)  return (1);
        else return (n * fact(n-1));
}
```

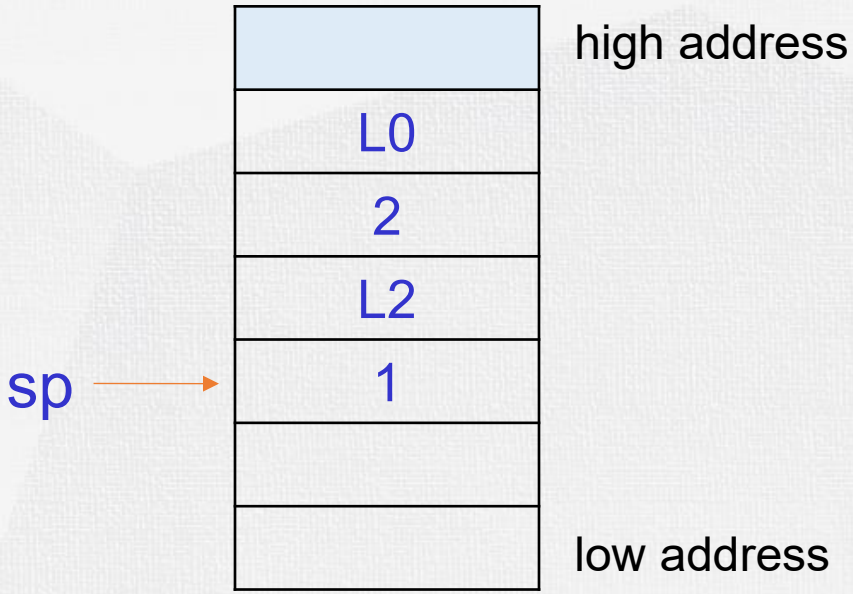


Non-Leaf Procedure Example

```
fact:
    addi    sp, sp, -8
    sw      ra, 4(sp)
    sw      a0, 0(sp)
    slti    t0, a0, 1
    beq     t0, zero, L1
    addi    a0, zero, 1
    addi    sp, sp, 8
    jr      ra
L1:
    addi    a0, a0, -1
    jal     fact
L2: addi    t1, a0, 0
    lw      a0, 0(sp)
    lw      ra, 4(sp)
    addi    sp, sp, 8
    mul     a0, a0, t1
    jr      ra
```

a0=0 t0=0 ra=L2

```
int  fact  (int n)
{
    if (n < 1)  return (1);
        else return (n * fact(n-1));
}
```

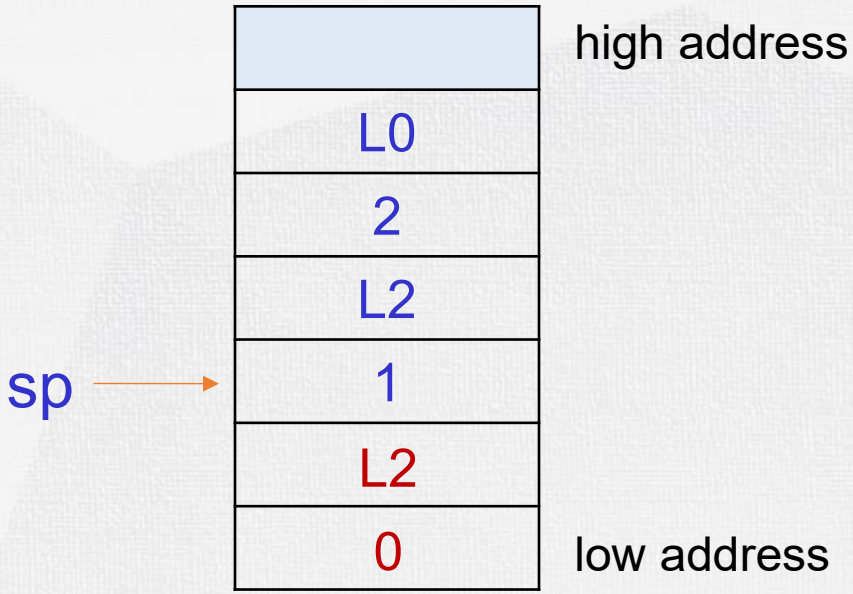


Non-Leaf Procedure Example

```
fact:
    addi    sp, sp, -8
    sw      ra, 4(sp)
    sw      a0, 0(sp)
    slti    t0, a0, 1
    beq     t0, zero, L1
    addi    a0, zero, 1
    addi    sp, sp, 8
    jr      ra
L1:
    addi    a0, a0, -1
    jal     fact
L2: addi    t1, a0, 0
    lw      a0, 0(sp)
    lw      ra, 4(sp)
    addi    sp, sp, 8
    mul     a0, a0, t1
    jr      ra
```

a0=0 t0=1 ra=L2

```
int  fact  (int n)
{
    if (n < 1)  return (1);
        else return (n * fact(n-1));
}
```

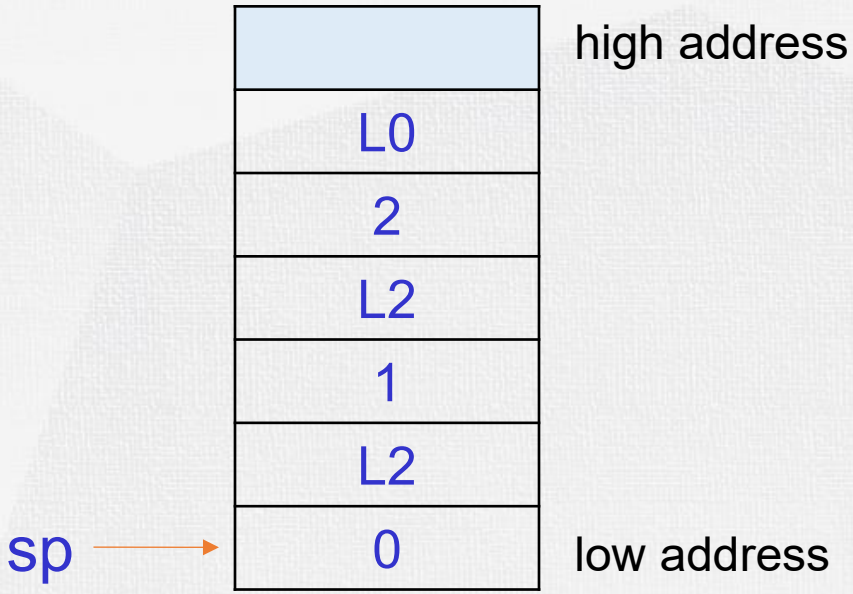


Non-Leaf Procedure Example

```
fact:
    addi    sp, sp, -8
    sw      ra, 4(sp)
    sw      a0, 0(sp)
    slti    t0, a0, 1
    beq     t0, zero, L1
    addi    a0, zero, 1
    addi    sp, sp, 8
    jr      ra
L1:
    addi    a0, a0, -1
    jal     fact
L2: addi    t1, a0, 0
    lw      a0, 0(sp)
    lw      ra, 4(sp)
    addi    sp, sp, 8
    mul     a0, a0, t1
    jr      ra
```

a0=1 t0=1 ra=L2

```
int   fact  (int n)
{
    if (n < 1)  return (1);
        else return (n * fact(n-1));
}
```

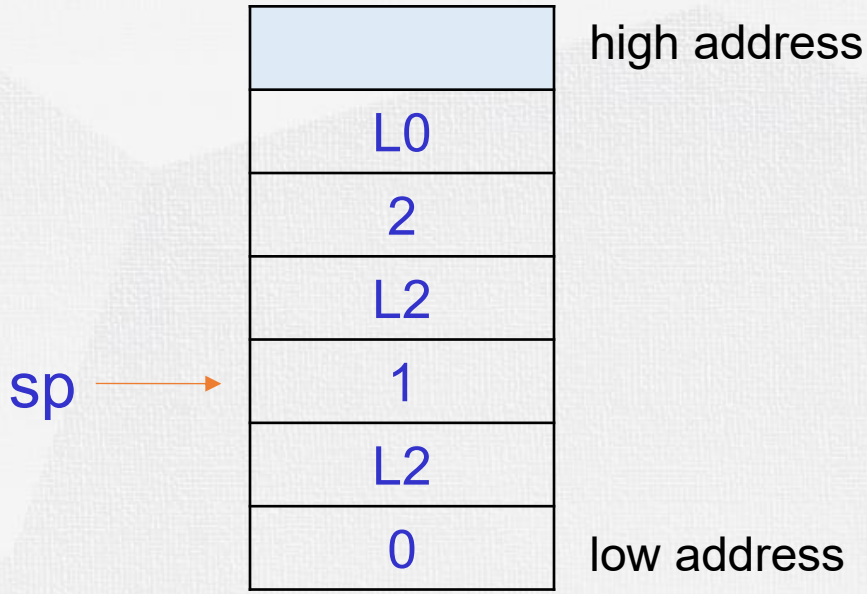


Non-Leaf Procedure Example

```
fact:
    addi    sp, sp, -8
    sw      ra, 4(sp)
    sw      a0, 0(sp)
    slti    t0, a0, 1
    beq     t0, zero, L1
    addi    a0, zero, 1
    addi    sp, sp, 8
    jr      ra
L1:
    addi    a0, a0, -1
    jal     fact
L2: addi    t1, a0, 0
    lw      a0, 0(sp)
    lw      ra, 4(sp)
    addi    sp, sp, 8
    mul     a0, a0, t1
    jr      ra
```

a0=1 t0=1 ra=L2

```
int  fact  (int n)
{
    if (n < 1)  return (1);
        else return (n * fact(n-1));
}
```

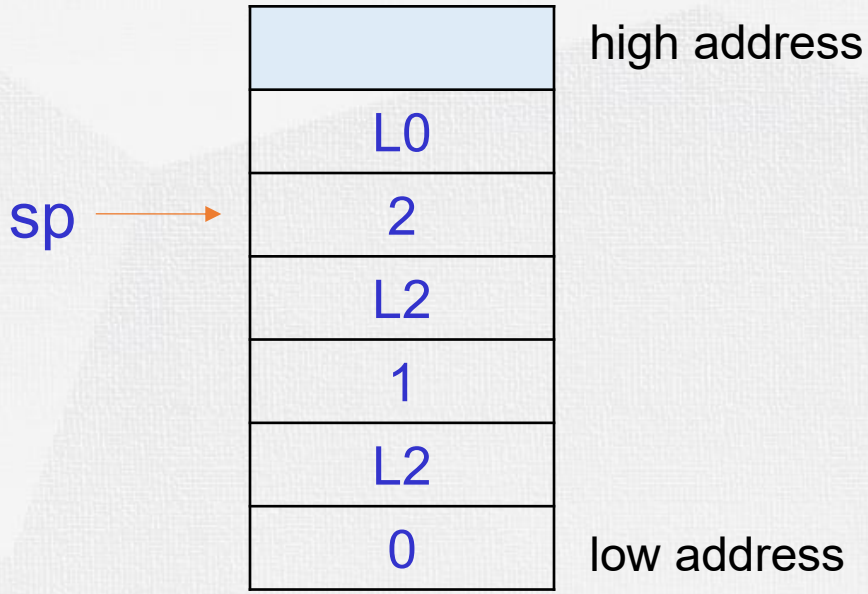


Non-Leaf Procedure Example

```
fact:
    addi    sp, sp, -8
    sw      ra, 4(sp)
    sw      a0, 0(sp)
    slti    t0, a0, 1
    beq     t0, zero, L1
    addi    a0, zero, 1
    addi    sp, sp, 8
    jr      ra
L1:
    addi    a0, a0, -1
    jal     fact
L2: addi    t1, a0, 0
    lw      a0, 0(sp)
    lw      ra, 4(sp)
    addi    sp, sp, 8
    mul     a0, a0, t1
    jr      ra
```

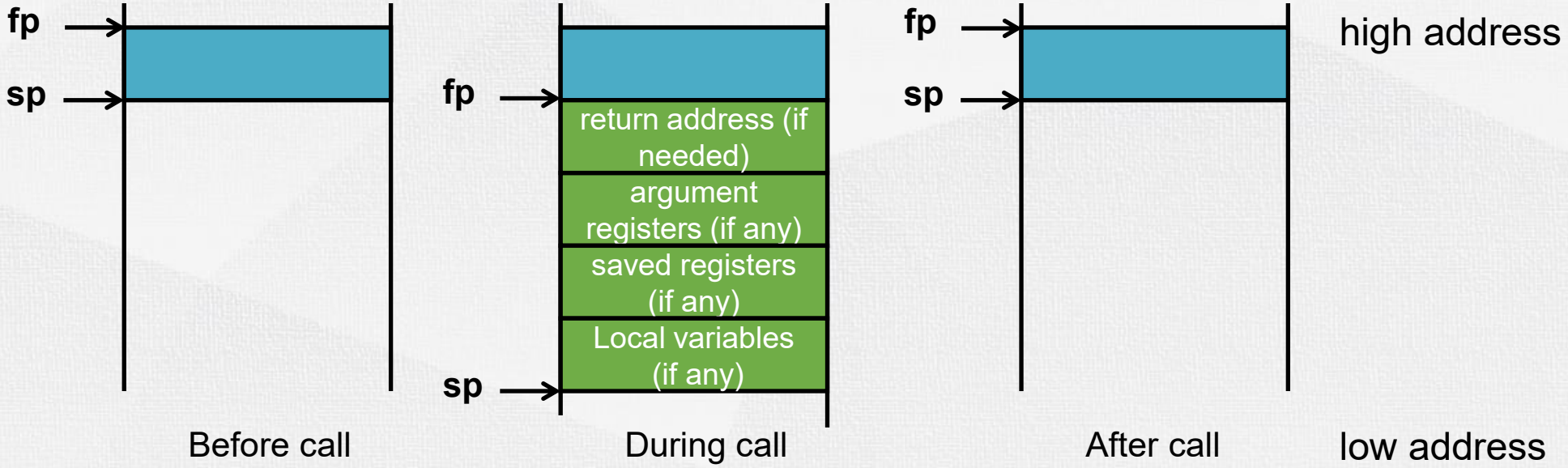
a0=2 t0=1 ra=L0

```
int  fact  (int n)
{
    if (n < 1)  return (1);
        else return (n * fact(n-1));
}
```



Procedure Frame

- Procedure frame (activation record)
  - Used by some compilers to manage stack storage
- frame pointer(fp) points to the start of the record
- stack pointer(sp) points to the end-variable addresses and may change during the execution of the procedure



Summary

- Functions called with jal, return with jalr (jr)
- The stack is your friend: Use it to save anything you need. Just leave it the way you found it!
- Instructions we know so far…
  - Arithmetic
  - Memory
  - Conditional Branches
  - Unconditional Branches (Jumps)
- Registers